Spatial Filtering & Convolution

Week 3 — Session 1: Convolution & Smoothing

Practical Image Processing

Lecture 3.1 ▪ 60–70 min

Recap & today

- Wk2: point ops look at *one* pixel at a time. They can brighten, threshold, stretch contrast — but they can't blur, sharpen, or find edges.
- Today (Session 1): the **neighbourhood**. A small **kernel** slides over the image and combines each pixel with its neighbours. This is **convolution**.
- We build convolution from scratch, then use it for our first job: **smoothing**.

Why it matters

Smoothing removes noise before every later stage; and convolution is the exact operation a CNN learns in Week 13.

Roadmap — Session 1

From point ops to neighbourhoods

Convolution

Smoothing

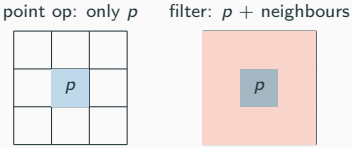
In-class exercise

Wrap-up

From point ops to neighbourhoods

The leap: one pixel → a window

- A **point operation** is a function of a single value: $g(x, y) = T\big(f(x, y)\big)$.
- A **spatial filter** is a function of a whole **neighbourhood** around (x, y).
- That is what lets us reason about *how a pixel differs from its surroundings* — the basis of blur, sharpen and edges.

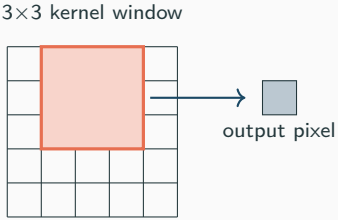


Key idea

Point ops answer "what is this pixel?". Filters answer "how does this pixel compare to the ones around it?".

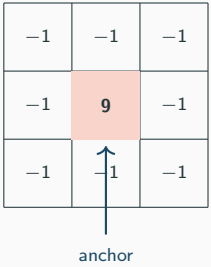
Convolution

A kernel slides over the image



For each pixel: multiply the neighbourhood by the kernel, sum, write the result. Slide and repeat.

Anatomy of a kernel



- **Weights**: the numbers that multiply each neighbour.
- **Anchor** (centre): where the result is written; usually the middle of an odd-sized kernel.
- **Support / size**: how many pixels the kernel touches (3×3, 5×5, ...).
- **Sum** of weights: 1 for a brightness-preserving blur, 0 for an edge detector.

Key idea

Every filter this week is just these four choices: weights, anchor, size, sum.

Convolution, formally

Math

$$g(x,y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} h(i,j)\, f(x-i,\, y-j)$$

- f = image, h = kernel (a.k.a. filter/mask), g = output; a 3×3 kernel has $k = 1$.
- The kernel's values *define* the operation: average $\rightarrow$ blur; differences $\rightarrow$ edges.
- Convolution flips the kernel (the $-i, -j$); **correlation** doesn't. For symmetric kernels they're identical.

Convolution vs correlation — the flip

Correlation lays the kernel down as-is:

$$g(x,y) = \sum_{i,j} h(i,j)\, f(x+i,\, y+j)$$

Convolution rotates the kernel 180° first (the $f(x-i, y-j)$).

- For *symmetric* kernels (box, Gaussian) the flip changes nothing.
- It matters for asymmetric kernels (gradients, Week 5).
- `cv2.filter2D` actually does *correlation*; flip the kernel yourself if you need true convolution.

Note

Convolution is associative and commutative; correlation is not. That algebra is why we prefer convolution in theory even when the code does correlation.

Two example kernels

Box blur (average):

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Each output = mean of its 3×3 neighbourhood.

Sharpen:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Boosts the centre relative to its neighbours (Session 2).

Key idea

The kernel *is* the algorithm. Change the numbers, change the effect — nothing else.

Worked example: one output pixel

Image patch P and box kernel K:

$$P = \begin{bmatrix} 20 & 40 & 60 \\ 80 & 100 & 120 \\ 140 & 160 & 180 \end{bmatrix}$$

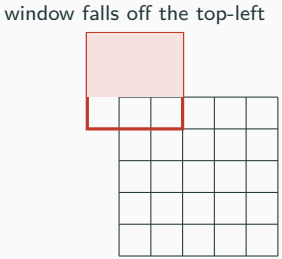
$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Element-wise multiply, then sum

$$\frac{1}{9}(20+40+60+80+100+120+140+160+180)$$
$$= \frac{900}{9} = \mathbf{100}$$

The centre becomes the *mean* (100). Because the patch is a smooth ramp, the mean equals the centre value — a flat gradient survives a blur.

Edge handling: the kernel hangs off the image



- **Zero-pad**: treat outside as 0 — can darken borders.
- **Replicate**: repeat the edge pixel (`BORDER_REPLICATE`).
- **Reflect**: mirror the image across the border (`BORDER_REFLECT`) — usually the most natural.

Normalization: keep the brightness

- A blur kernel should **sum to 1** — it preserves average brightness.
- An edge kernel sums to 0 — flat regions map to 0, only changes survive.
- Kernel not normalized $\rightarrow$ the image gets darker/brighter as a side effect.

✕ Pitfall

Forgetting to normalize a blur kernel (dividing by 9 for a 3×3) is why a "smoothed" image sometimes comes out too dark.

Convolution from scratch (NumPy)

```python
import numpy as np
def convolve3x3(img, k):
    k = np.flipud(np.fliplr(k))          # flip -> true convolution
    p = np.pad(img, 1, mode="reflect")   # reflect border
    out = np.zeros_like(img, dtype=float)
    for y in range(img.shape[0]):
        for x in range(img.shape[1]):
            region = p[y:y+3, x:x+3]     # 3x3 neighbourhood
            out[y, x] = (region * k).sum()
    return out
```

This is the whole idea in nine lines. Later you replace the Python loops with `cv2.filter2D` for speed.

Padding, numerically

Original row near the left border:

$$\ldots \mid \mathbf{40}\ \ 60\ \ 80\ \ldots$$

- **zero**: $0 \mid 40\ \ 60$
- **replicate**: $40 \mid 40\ \ 60$
- **reflect**: $60 \mid 40\ \ 60$

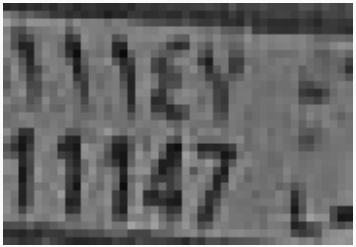
- Zero-padding invents a dark neighbour that was never there $\rightarrow$ border darkening.
- Replicate/reflect reuse real data, so a blur near the edge behaves like the interior.
- Reflect avoids the flat "smear" that replicate can cause on a strong border.

✓ Good practice

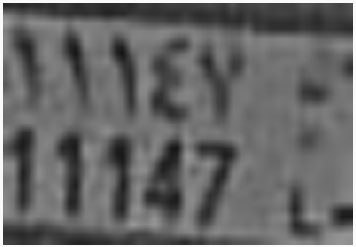
Default to `BORDER_REFLECT` for smoothing; it keeps edge pixels honest.

Smoothing

Box vs Gaussian blur — on a real plate



original



box blur 5×5



Gaussian σ=1.5

Box averages equally (boxy artifacts); Gaussian weights the centre more (smoother, natural).

The Gaussian kernel

Math

$$h(i,j) = \frac{1}{2\pi\sigma^2}\, e^{-(i^2+j^2)/(2\sigma^2)}$$

- σ controls the blur amount; larger σ = more smoothing, wider kernel.
- **Separable**: a 2D Gaussian = a 1D horizontal blur then a 1D vertical blur $\rightarrow$ much faster.
- The go-to smoother: removes noise while preserving structure better than a box.

Building a 3×3 Gaussian by hand

Sample $e^{-(i^2+j^2)/2}$ at $\sigma = 1$ and round to a nice integer stencil:

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Weights fall off from the centre; they sum to 16, so we divide by 16.

- Centre weight 4 = most influence; corners 1 = least.
- Sum-to-1 (after $\div 16$) keeps brightness.
- It is the outer product of $\frac{1}{4}[1, 2, 1]$ with itself — proof it is **separable**.

Key idea

A Gaussian is just a box that trusts the centre more than the corners.

Why smooth at all?

- Remove noise before thresholding/edges (noise creates false edges — Week 5).
- Blur *before* downsampling to avoid aliasing (Week 1's Nyquist).
- Suppress fine texture so large structures dominate (find the page, not the paper grain).

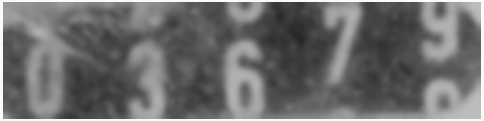
✕ Pitfall

But blur also destroys detail. Too much and the digits smear together — a trade-off you tune.

Smoothing the water-meter image



raw meter crop



Gaussian σ=1.5

- Sensor speckle and printing texture vanish; the digit *shapes* stay.
- This is the pre-processing step before we threshold the odometer digits in the DocScan capstone.
- Pick σ big enough to kill grain, small enough to keep the strokes.

✓ Good practice

Smooth first, then threshold. It is the single most reliable habit for clean digit masks.

Choosing the kernel size vs σ

- Rule of thumb: kernel radius $\approx 3\sigma$, so a $\sigma = 1.5$ Gaussian wants a $\approx 9 \times 9$ window.
- Too small a window *clips* the Gaussian tails — you lose the smoothness you paid for.
- In OpenCV, pass `ksize=(0,0)` and let it derive the size from σ (or vice-versa).

Note

Odd kernel sizes $(3, 5, 7, \dots)$ keep a well-defined centre pixel. Even sizes shift the image by half a pixel.

Blur strength on a pixel grid

A tiny image with one hot pixel:

20	20	20	20
20	20	240	20
20	20	20	20

After a 3×3 box blur:

20	45	45	45
20	45	45	45
20	45	45	45

The single value 240 is spread across its neighbours ($(240 + 8 \cdot 20)/9 \approx 45$); the outlier's energy is shared, so the peak drops and the surroundings rise.

Smoothing in code

```
box    = cv2.blur(img, (5, 5))              # simple average
gauss  = cv2.GaussianBlur(img, (7, 7), 1.5) # sigma = 1.5
gauss2 = cv2.GaussianBlur(img, (0, 0), 2.0) # size derived from sigma
# custom kernel via the general tool:
k = np.ones((3, 3), np.float32) / 9.0
box2   = cv2.filter2D(img, -1, k)
```

`cv2.filter2D` convolves with *any* kernel — the general tool you fall back to for custom filters.

In-class exercise

In-class exercise (10 min)

Blur by hand

A 3×3 patch with one bright speck, and the box kernel:

$$P = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 250 & 10 \\ 10 & 10 & 10 \end{bmatrix} \qquad K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

1. Compute the box-blur output for the centre pixel.
2. By how much did the speck's value drop? What does that tell you about box blur and noise?
3. Would a $\frac{1}{16}[1, 2, 1; 2, 4, 2; 1, 2, 1]$ Gaussian give a higher or lower centre value? Why?

Debrief

- Box: $(10{\cdot}8 + 250)/9 = 330/9 \approx \mathbf{37}$. The speck (250) was crushed to $\sim$37 — noise *averaged away*.
- So a box blur is a cheap denoiser: it spreads any single bright/dark outlier across its neighbours.
- Gaussian: centre weight $4/16$, so $(10{\cdot}12 + 250{\cdot}4)/16 = 1120/16 = \mathbf{70}$ — *higher*, because the Gaussian trusts the centre more, so it removes less of the speck.

Key idea

More centre weight $\Rightarrow$ gentler smoothing. That is the box-vs-Gaussian trade-off in one number.

Wrap-up

What you should now be able to do

- Explain convolution and how a kernel defines an operation.
- Convolve a patch by hand and write a from-scratch 3×3 convolution.
- Handle borders (zero/replicate/reflect) and normalize a blur kernel.
- Smooth with box vs Gaussian, choose σ and kernel size, and know the detail-vs-noise trade-off.

Next session

Session 2 — Sharpening, nonlinear & advanced filtering

- Sharpening: add the detail back with the **unsharp mask** — and the halos it causes.
- Nonlinear filters: **median** (kills salt-and-pepper) and **bilateral** (smooth but keep edges).
- The **low-pass / high-pass** view, **separability** for speed, and the bridge to CNNs.
- Then this week's deliverable.

Bring your from-scratch convolution — Session 2 builds directly on it.

The kernel is the algorithm.

Today: convolution, and its first job — smoothing.